Chapitre 3 — Conditions et logique

Prérequis : chapitres 1 et 2 · **Exercices :** `E03_Conditions_et_logique`

Objectifs

- écrire des conditions avec les opérateurs de comparaison et les opérateurs logiques ;
- utiliser `if`, `else if`, `else` et `switch` ;
- vérifier qu'une saisie a réussi et que la valeur appartient à un domaine autorisé ;
- signaler une erreur avec `std::cerr` et un code de retour.

Sommaire

- Chapitre 3 — Conditions et logique
  - Objectifs
  - Sommaire
  - 1. Comparaisons et opérateurs logiques
  - 2. La structure `if`
  - 3. La structure `switch`
  - 4. Vérifier une saisie et un domaine de valeurs
  - Erreurs fréquentes
  - Synthèse

1. Comparaisons et opérateurs logiques

Une comparaison produit un booléen : `true` ou `false`.

Opérateur	Signification	Exemple
`==`	Égal à	`a == b`
`!=`	Différent de	`a != b`
`<`	Inférieur à	`a < b`
`>`	Supérieur à	`a > b`
`<=`	Inférieur ou égal à	`a <= b`
`>=`	Supérieur ou égal à	`a >= b`

> **Ne pas confondre :** `=` **affecte** une valeur ; `==` **compare** deux valeurs.

```cpp
int nombre{10};

nombre = 5;                  // affectation
bool egal{nombre == 5};      // comparaison : true
```

Opérateur	Signification	Vrai si…
`&&`	ET logique	les deux conditions sont vraies
`||`	OU logique	au moins une condition est vraie
`!`	NON logique	la condition est fausse

```cpp
int temperature{25};

bool temperatureNormale{temperature >= 0 && temperature <= 40};   // true
bool temperatureAnormale{temperature < 0 || temperature > 40};    // false
bool horsNormes{!temperatureNormale};                              // false
```

2. La structure `if`

```cpp
int temperature{25};

if (temperature < 0) {
    std::cout << "Temperature negative\n";
} else if (temperature <= 30) {
    std::cout << "Temperature normale\n";
} else {
    std::cout << "Temperature elevee\n";
}
```

Les conditions sont testées **dans l'ordre** ; dès qu'une est vraie, son bloc est exécuté et les autres branches sont ignorées. Ici, la deuxième condition n'a pas besoin de tester `temperature >= 0` : si on l'atteint, c'est que la première était fausse.

> **Conseil :** on utilise **toujours des accolades**, même pour une seule instruction.

3. La structure `switch`

`switch` compare **une même expression entière** (ou un caractère) à plusieurs valeurs **constantes**.

```cpp
int choix{2};

switch (choix) {
    case 1:
        std::cout << "Demarrage\n";
        break;
    case 2:
        std::cout << "Arret\n";
        break;
    default:
        std::cout << "Choix invalide\n";
        break;
}
```

> **Attention :** sans `break`, l'exécution continue dans le `case` suivant.

`default` traite les valeurs non prévues. `switch` ne fonctionne ni avec une `std::string` ni avec un `double`.

Pour **déclarer une variable** dans un `case`, on entoure les instructions de ce `case` d'accolades. Sans elles, g++ refuse de compiler (`jump to case label` : la variable serait visible dans les `case` suivants sans avoir été initialisée).

```cpp
switch (choix) {
    case 1: {
        double tension{12.0};          // variable locale au bloc du case 1
        std::cout << tension << '\n';
        break;
    }
    default:
        std::cout << "Choix invalide\n";
        break;
}
```

4. Vérifier une saisie et un domaine de valeurs

Une saisie correcte franchit **deux contrôles** :

1. la **lecture** a réussi : l'utilisateur a bien tapé un nombre ;
2. la valeur est dans le **domaine** autorisé (cahier des charges, documentation du capteur).

```cpp
#include <iostream>

int main() {
    constexpr int TEMPERATURE_MIN{-40};
    constexpr int TEMPERATURE_MAX{85};

    int temperature{};

    std::cout << "Temperature : ";

    if (!(std::cin >> temperature)) {
        std::cerr << "Erreur : saisie non numerique.\n";
        return 1;
    }

    if (temperature < TEMPERATURE_MIN || temperature > TEMPERATURE_MAX) {
        std::cerr << "Erreur : temperature hors limites.\n";
        return 1;
    }

    std::cout << "Mesure acceptee.\n";

    return 0;
}
```

- `std::cin >> temperature` peut être utilisé comme une condition : elle est **vraie** si la lecture a réussi. Si l'utilisateur tape `abc`, elle est fausse, donc `!(std::cin >> temperature)` est vraie.
- `std::cerr` est le **flux d'erreur** standard. Il s'utilise comme `std::cout`, mais il est réservé aux messages d'erreur ; ses messages s'affichent même quand la sortie normale est redirigée vers un fichier.
- Dans `main()`, `return 1;` arrête le programme avec un **code de retour non nul**, qui signale un échec à celui qui a lancé le programme (`return 0;` signale une fin normale). Dans le terminal, on affiche le code de la dernière commande avec `echo $?` (Linux, macOS, Git Bash) ou `$LASTEXITCODE` (PowerShell).

Remarque : après une lecture ratée, `std::cin` reste en **état d'échec** : toutes les lectures suivantes échouent aussi. Ici, le programme s'arrête aussitôt, ce qui évite le problème. Pour **redemander** la valeur, il faut d'abord réparer le flux : c'est l'objet du chapitre 4 (§6).

Erreurs fréquentes

Erreur	Effet	Correction
`if (x = 5)`	Affecte 5 ; condition toujours vraie	`if (x == 5)` (g++ `-Wall` le signale)
`0 <= x <= 40`	Condition toujours vraie	`x >= 0 && x <= 40`
`&` au lieu de `&&`	Opération bit à bit (chapitre 6)	`&&` pour combiner des conditions
`break` oublié	Exécution du `case` suivant	Un `break` par `case`
Variable déclarée dans un `case` sans accolades	Erreur `jump to case label`	Entourer le `case` d'accolades
Saisie non vérifiée	Calcul sur une valeur fausse	Tester `std::cin >> x`, puis le domaine

Synthèse

- Une comparaison (`==`, `!=`, `<`, `>`, `<=`, `>=`) produit un `bool` ; `&&`, `||` et `!` combinent des conditions.
- Dans une plage : `x >= min && x <= max` ; hors plage : `x < min || x > max`.
- `if / else if / else` teste les conditions dans l'ordre et n'exécute qu'une branche ; accolades toujours.
- `switch` compare un entier ou un caractère à des constantes ; un `break` par `case`, un `default`.
- Une saisie se contrôle en deux temps : lecture réussie, puis valeur dans le domaine.
- En cas d'erreur : message sur `std::cerr` et `return 1;` dans `main()`.